

ELO312 Laboratorio de Estructura de Computadores

Sesión Guiada 2: GPIO

Rodrigo Jiménez, Mauricio Solís, Alejandro Weinstein
20260816

Esta sesión guiada tiene dos objetivos. El primer objetivo es familiarizarlo con el entorno de desarrollo que utilizará en el laboratorio, basado en la tarjeta Nucleo [2], en STM32CubeMX [5] y en CubeIDE [4]. El segundo objetivo es conocer los elementos básicos del módulo *General Purpose Input Output* (GPIO) del microcontrolador STM32L476RG.

1. Manejo de los registros de GPIO

Cree un nuevo proyecto para la tarjeta de desarrollo y realice los siguientes ejercicios apoyándose en el capítulo 8 del manual [3].

1.1. Control de estado del LED built-in de la tarjeta

La tarjeta de desarrollo tiene un LED verde conectado al pin PA5 (puerto A, pin 5). Cree un proyecto en STM32CubeIDE y utilice el archivo `template_1.c` (disponible en el repositorio GitHub del curso) como parte del contenido de su función `main.c`. Agregue las líneas de código necesarias para encender y apagar el LED mediante el control del registro del puerto correspondiente, utilizando el *debug* paso a paso (no necesita retardos, pues es paso a paso). Sírvese de los comentarios del código como guía.

1.2. Lectura de estado del pulsador azul built-in de la tarjeta

La tarjeta de desarrollo también posee un pulsador azul conectado directamente al pin PC13 (puerto C, pin 13). El pulsador es del tipo normalmente abierto y está conectado a una resistencia pull-up. Utilice el archivo `template_2.c` (disponible en el repositorio GitHub del curso) como parte del contenido de su función `main.c`. Puede usar el mismo proyecto anterior o uno nuevo. Agregue las líneas de código necesarias para guardar en una variable el estado del pulsador mediante el control del registro del puerto correspondiente, utilizando el *debug* paso a paso. Sírvese de los comentarios del código como guía.

2. Control del LED a través del pulsador usando STM32CubeMX y HAL

De ahora en adelante, debe utilizar el código base generado por STM32CubeMX, basado en la capa de software provista por ST, conocida como *Hardware Abstraction Layer* (HAL) [1]. Es importante que respete las reglas que esto implica. Procure escribir su código dentro de los comentarios del tipo

```
1 /* USER CODE BEGIN ... */
2 ...
3 ...
4 /* USER CODE END ... */
```

Tenga en cuenta que dichos espacios son creados por la herramienta STM32CubeMX y que, cada vez que se reconstruya el código, se respaldarán, lo que garantiza que el código de usuario no se pierda. Si no está seguro de dónde colocar su código, pregunte al *staff*.

Desarrolle una aplicación con las siguientes características:

- Debe implementar una máquina de estados con dos estados: en el primer estado (caso por defecto), el LED debe estar constantemente encendido, y en el segundo estado, el LED debe parpadear a una frecuencia a elección (debe apreciarse a simple vista). Se recomienda utilizar la función `HAL_Delay(...)` para esta actividad, que genera una espera activa durante un tiempo determinado. ¿Qué significa “espera activa”? El procesador está ocupado mientras durante esa cantidad de tiempo
- La transición de estados debe realizarse con el canto de subida (o bajada, pero no ambos) del pulsador azul.

Procure trabajar de forma incremental: primero, compruebe que puede leer el pulsador azul; luego, implemente un detector de canto de subida (o bajada, pero no ambos cantos) por software; y, finalmente, ocupe esta detección para moverse en su máquina de estados.

3. Cambiar la frecuencia de parpadeo del LED

Desarrolle una nueva aplicación con las siguientes características:

- Debe utilizar el LED de la tarjeta de desarrollo para que parpadee.
- La frecuencia de parpadeo debe ser variable. Utilice un arreglo para definir al menos 3 retardos a modo de generar al menos 3 frecuencias distintas (de menor a mayor).
- Tras cada canto de subida (o bajada) del pulsador azul, la frecuencia debe cambiar de acuerdo con los retardos del arreglo.
- Una vez recorrido el arreglo, debe devolverse en orden inverso (no debe volver al primer elemento). Por ejemplo si tenemos 3 retardos, el orden debe ser: $R1 \rightarrow R2 \rightarrow R3 \rightarrow R2 \rightarrow R1 \rightarrow R2 \rightarrow R3 \dots$

4. Uso de GPIO con hardware externo

Durante la sesión, tendrá a su disposición una tarjeta con pulsadores, LEDs e interruptores (ver la figura 1). Como primer paso, realice el proceso de ingeniería inversa para obtener el esquemático de esta tarjeta. Luego, llame a algún miembro del *staff* y explique, usando este esquemático, cómo entiende las conexiones de la tarjeta.

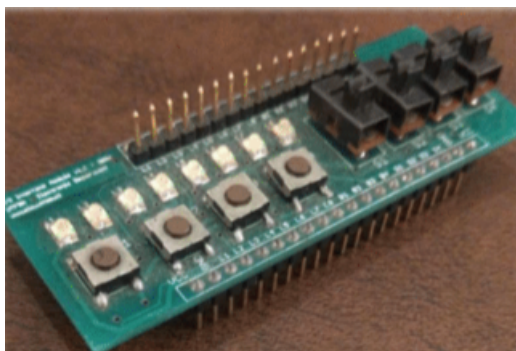


Figura 1: Tarjeta PCB de entrada/salida.

Ayuda: si nunca ha trabajado con hardware de este estilo, pruebe encendiendo los LEDs directamente con 3.3 V y luego con los interruptores alimentados con 3.3 V.

Considere, para esta actividad, 8 LEDs y 2 interruptores de dicha tarjeta conectados a la tarjeta Nucleo. Desarrolle un programa con las siguientes características:

- Debe utilizar 8 pines como salidas (para controlar LEDs externos).
- Debe utilizar 2 pines como entradas (para leer los interruptores externos).
- También debe utilizar el pulsador azul de la tarjeta como entrada.
- Debe inicializar los 8 LEDs externos con el patrón 11110000, donde un 1 indica que el LED está encendido y un 0 que está apagado.
- Dependiendo del estado de los interruptores, debe realizar lo siguiente:
 - Caso 00: Debe hacer parpadear los LEDs que antes estaban encendidos a una frecuencia arbitraria (distinguible para el ojo humano, use `#define` para definir una constante de retardo). Asegúrese de que los LEDs queden encendidos antes de cambiar de caso.
 - Caso 01: Los LEDs se desplazan a la derecha (como una rotación, es decir, luego de 00001111 viene 10000111) a la misma frecuencia que en el caso anterior.
 - Caso 10: Los LEDs se desplazan a la izquierda (como una rotación, es decir, luego de 11110000 viene 11100001) a la misma frecuencia que en el caso anterior.
 - Caso 11: Todos los LEDs parpadean simultáneamente a la misma frecuencia que en el caso anterior. Asegúrese de que su programa recuerde cuáles eran los 4 LEDs encendidos antes de que cayera en este estado.
- El canto de subida (o bajada) del pulsador azul actuará como el PLAY/STOP de la máquina anterior. Es decir, comenzando en STOP, al presionar el pulsador azul (START), debe ejecutarse la máquina de estados, actuando según las indicaciones de arriba. Si se presiona nuevamente el pulsador (STOP), los LEDs quedan como estaban después de ejecutar la máquina por última vez.

Referencias

- [1] ST. Hardware Abstraction Layer. <https://tinyurl.com/3wakxmf4>, 2024. [Accessed 17-08-2026].
- [2] ST. Nucleo-L476RG. <https://www.st.com/en/evaluation-tools/nucleo-l476rg.html>, 2024. [Accessed 17-08-2026].
- [3] ST. RM0351 reference manual. <https://tinyurl.com/4355ebuh>, 2024. [Accessed 17-08-2026].
- [4] ST. STM32CubeIDE. <https://www.st.com/en/development-tools/stm32cubeide.html>, 2024. [Accessed 17-08-2026].
- [5] ST. STM32CubeMX. <https://www.st.com/en/development-tools/stm32cubemx.html>, 2025. [Accessed 17-08-2026].